

Lab 05 — Predição de Preços de Imóveis

Documentação da Solução Vencedora

2026

Contents

1	Visão Geral	2
2	Pré-processamento	2
2.1	Remoção de Outliers	2
2.2	Transformação da Variável Alvo	2
2.3	Imputação Contextual	3
3	Feature Engineering	3
3.1	Agregações de Área	3
3.2	Flags Binárias de Presença	3
3.3	Interações Multiplicativas	4
3.4	Transformações Logarítmicas e Temporais	4
4	Encoding de Variáveis Categóricas	4
5	Normalização	5
6	Modelos Base	5
7	Validação Cruzada (Out-of-Fold)	5
8	Meta-Modelo (Stacking)	6
9	Resultado Final	6
10	Resumo das Decisões de Design	7

1 Visão Geral

O problema consiste em prever o preço de venda de imóveis residenciais (dataset Ames Housing) a partir de 79 variáveis descritivas. A métrica de avaliação é o RMSE calculado sobre $\log(\text{SalePrice})$.

Métrica	Baseline (regressão linear)	Solução final
RMSE_LOG OOF	~0.23	0.1140
Posição	—	1º lugar

A solução é um **stacking ensemble** de dois níveis: cinco modelos de gradient boosting como camada base e um blend de RidgeCV + ElasticNet como meta-modelo.

2 Pré-processamento

2.1 Remoção de Outliers

Antes de qualquer modelagem, quatro filtros são aplicados ao conjunto de treino para eliminar observações atípicas que distorceriam o aprendizado:

```
train_df = train_df[train_df["GrLivArea"] < 4500]
train_df = train_df[~((train_df["GrLivArea"] > 4000) & (train_df["SalePrice"] < 20000))
train_df = train_df[train_df["LotArea"] < 100000]
train_df = train_df[train_df["TotalBsmtSF"] < 6000]
```

O segundo filtro é especialmente importante: remove casas grandes vendidas a preço baixo, que são anomalias conhecidas do dataset Ames e prejudicam modelos de árvore.

2.2 Transformação da Variável Alvo

O target `SalePrice` é transformado com `log1p` antes do treino:

```
y = np.log1p(train_df["SalePrice"])
```

Isso aproxima a distribuição da normal, reduz a influência de valores extremos e é o formato esperado na submissão — as predições são entregues em escala logarítmica, sem reverter com `expm1`.

2.3 Imputação Contextual

Em vez de imputação genérica por mediana, os valores ausentes são tratados com base no domínio do problema:

- **Catégoricas de ausência semântica** (ex: `GarageType`, `PoolQC`): NaN significa que o imóvel não possui aquela característica, portanto preenchidos com a string `"None"`.
- **Numéricas de área ou contagem** (ex: `GarageArea`, `BsmtFinSF1`): imóveis sem garagem ou porão têm área zero, portanto preenchidos com 0.
- **Demais numéricas**: imputação por mediana dentro de cada fold de validação cruzada, usando `SimpleImputer`.

3 Feature Engineering

A função `add_features` expande o conjunto original de 80 para 103 features, agrupadas em quatro categorias:

3.1 Agregações de Área

```
df["TotalSF"] = df["TotalBsmtSF"] + df["1stFlrSF"] + df["2ndFlrSF"]
df["TotalPorchSF"] = df["OpenPorchSF"] + df["EnclosedPorch"] + ...
df["TotalBathrooms"] = df["FullBath"] + 0.5*df["HalfBath"] + ...
```

Capturam a área utilizável total, que é mais preditiva que qualquer área individual.

3.2 Flags Binárias de Presença

```
df["HasPool"] = (df["PoolArea"] > 0).astype(int)
df["HasGarage"] = (df["GarageArea"] > 0).astype(int)
df["HasBsmt"] = (df["TotalBsmtSF"] > 0).astype(int)
df["HasFireplace"] = (df["Fireplaces"] > 0).astype(int)
df["IsRemodeled"] = (df["YearRemodAdd"] != df["YearBuilt"]).astype(int)
df["IsNew"] = (df["YrSold"] == df["YearBuilt"]).astype(int)
```

Modelos de árvore beneficiam-se de splits binários explícitos quando a distinção presença/ausência é mais relevante que a magnitude.

3.3 Interações Multiplicativas

```
df["OverallQual_TotalSF"] = df["OverallQual"] * df["TotalSF"]
df["OverallQual_GrLivArea"] = df["OverallQual"] * df["GrLivArea"]
df["OverallQual_sq"] = df["OverallQual"] ** 2
df["QualCondScore"] = df["OverallQual"] * df["OverallCond"]
```

OverallQual é a variável mais correlacionada com preço. Suas interações com área capturam o efeito não-linear de que qualidade importa proporcionalmente mais em casas grandes.

3.4 Transformações Logarítmicas e Temporais

```
df["LotAreaLog"] = np.log1p(df["LotArea"])
df["GrLivAreaLog"] = np.log1p(df["GrLivArea"])
df["HouseAge"] = df["YrSold"] - df["YearBuilt"]
df["RemodAge"] = df["YrSold"] - df["YearRemodAdd"]
```

Log-transforms reduzem a skewness de variáveis de área, ajudando especialmente o meta-modelo linear. As features de idade capturam depreciação e o efeito de reformas recentes.

4 Encoding de Variáveis Categóricas

As 44 variáveis categóricas são transformadas com **Target Encoding** dentro de cada fold:

```
enc = ce.TargetEncoder(cols=cat_cols, smoothing=10)
enc.fit(X_tr, y_tr)
```

O parâmetro `smoothing=10` é crítico: mistura a média da categoria com a média global proporcionalmente ao tamanho da categoria, evitando data leakage em categorias raras. O valor 10 (versus 0.3 do baseline) reduz significativamente o overfitting do encoding.

O encoder é sempre fitado apenas nos dados de treino do fold e aplicado em validação e teste, respeitando o isolamento de informação da validação cruzada.

5 Normalização

As variáveis numéricas são normalizadas com `RobustScaler` em vez de `StandardScaler`:

```
scaler = RobustScaler()
```

`RobustScaler` usa mediana e IQR em vez de média e desvio padrão, tornando-o insensível a outliers remanescentes após a filtragem inicial.

6 Modelos Base

Cinco modelos de gradient boosting com hiperparâmetros distintos formam a camada base do ensemble. A diversidade entre eles é intencional: erros não correlacionados se cancelam no stacking.

Modelo	n_estimators	learning_rate	num_leaves~/~depth	Característica
LGB1	6000	0.005	31	Conservador, balanceado
LGB2	8000	0.003	15	Raso, muito regularizado
LGB3	5000	0.008	63	Muitas folhas, agressivo
Cat	5000	0.007	6	Melhor OOF individual
XGB	5000	0.005	5	Complementa LGB/Cat

OOF RMSE individuais (seed 42):

- LGB1: 0.126874
- LGB2: 0.123090
- LGB3: 0.119819
- CatBoost: **0.116575** (melhor)
- XGBoost: 0.125426

7 Validação Cruzada (Out-of-Fold)

```
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

10 folds foram utilizados em vez dos 5 do baseline por dois motivos: cada modelo é avaliado em 10 conjuntos distintos, reduzindo a variância da estimativa de RMSE; e cada modelo de treino usa 90% dos dados em vez de 80%, gerando previsões mais fortes.

As previsões Out-of-Fold (OOF) são usadas como features para o meta-modelo, garantindo que o meta-modelo nunca veja previsões feitas sobre dados em que o modelo base foi treinado.

8 Meta-Modelo (Stacking)

As previsões OOF dos cinco modelos base formam uma matriz de entrada para dois meta-modelos:

```
# Meta 1
meta_ridge = RidgeCV(alphas=np.logspace(-4, 2, 50), cv=10)
meta_ridge.fit(X_meta, y)    # RMSE: 0.113588

# Meta 2
meta_en = ElasticNet(alpha=0.0003, l1_ratio=0.5, max_iter=50000)
meta_en.fit(X_meta, y)      # RMSE: 0.114635

# Blend final (50/50)
final = 0.5 * meta_ridge.predict(...) + 0.5 * meta_en.predict(...)
# RMSE: 0.113965
```

RidgeCV seleciona automaticamente o melhor α entre 50 valores via cross-validation interna. O blend 50/50 com ElasticNet reduz ligeiramente a variância da predição final em relação a usar apenas um dos dois.

9 Resultado Final

```
OOF RMSE FINAL (blend meta): 0.113965
OOF RMSE FINAL (média direta): 0.119427
→ Usando blend de metas (melhor)
```

A submissão contém os valores preditos diretamente em escala $\log_{1p}(\text{SalePrice})$, que é o formato esperado pela competição.

10 Resumo das Decisões de Design

Decisão	Alternativa descartada	Motivo
Imputação contextual por domínio	Mediana global	NaN em GarageArea significa 0, não
RobustScaler	StandardScaler	Mais estável com outliers remanesce
TargetEncoding smoothing=10	smoothing=0.3	Menos leakage em categorias raras
10-fold CV	5-fold	Menor variância, mais dados por tre
5 modelos heterogêneos	Apenas LightGBM	Erros não correlacionados melhoram
RidgeCV como meta	ElasticNet puro	Seleção automática de alpha, mais e
Predição em log scale	<code>expm1</code> antes de submeter	Formato exigido pela competição